Practical 12

Collections Framework — Milestone 2 checkpoint

Part A — Practice Programs (Hour 1)

1) Word Frequency — HashMap and Comparator

Code

```
import java.util.*;

public class WordFrequency {
    public static void main(String[] args) {
        String paragraph = "the quick brown fox jumps over the lazy dog the fox barks and the dog
            runs";
        Map<String, Integer> freq = new HashMap<>();
        for (String w : paragraph.split("\\s+")) {
            freq.put(w, freq.getOrDefault(w, 0) + 1);
        }
        List<Map.Entry<String, Integer>> entries = new ArrayList<>(freq.entrySet());
        entries.sort(new Comparator<Map.Entry<String, Integer>>() {
            public int compare(Map.Entry<String, Integer> a, Map.Entry<String, Integer> b) {
                int c = b.getValue() - a.getValue();          // highest frequency first
                return c != 0 ? c : a.getKey().compareTo(b.getKey());
            }
        });
        for (Map.Entry<String, Integer> e : entries) {
            System.out.println(e.getKey() + " : " + e.getValue());
        }
    }
}
```

Output

```
the : 4
dog : 2
fox : 2
and : 1
barks : 1
brown : 1
jumps : 1
lazy : 1
over : 1
quick : 1
runs : 1
```

2) Unique Visitors — HashSet and LinkedHashSet

Code

```
import java.util.*;

public class UniqueVisitors {
    public static void main(String[] args) {
        List<String> logins = List.of("riya", "aman", "riya", "neha", "aman", "karan", "neha",
            "riya");
        Set<String> unique = new HashSet<>(logins);
        Set<String> ordered = new LinkedHashSet<>(logins);
        System.out.println("Total logins: " + logins.size());
        System.out.println("Unique users (HashSet): " + unique.size());
        System.out.print("First three distinct (LinkedHashSet):");
        int shown = 0;
        for (String u : ordered) {
            if (shown++ == 3) break;
            System.out.print(" " + u);
        }
        System.out.println();
    }
}
```

Output

```
Total logins: 8
Unique users (HashSet): 4
First three distinct (LinkedHashSet): riya aman neha
```

3) Leaderboard (For Advanced Learners — home) — top-N sorted by score, ties by name

Code

```
import java.util.*;

public class Leaderboard {
    public static void main(String[] args) {
        Map<String, Integer> scores = new HashMap<>();
        scores.put("Riya", 2400);
        scores.put("Aman", 3100);
        scores.put("Neha", 2400);
        scores.put("Karan", 1800);
        scores.put("Dev", 3100);
        scores.put("Isha", 2750);

        int topN = 3;
        List<Map.Entry<String, Integer>> list = new ArrayList<>(scores.entrySet());
        // highest score first; ties broken by name (A-Z)
        list.sort(Map.Entry.<String, Integer>comparingByValue().reversed()
                .thenComparing(Map.Entry.comparingByKey()));

        System.out.println("Top " + topN + ":");
        for (int i = 0; i < topN && i < list.size(); i++) {
            System.out.println((i + 1) + ". " + list.get(i).getKey() + " - " +
                list.get(i).getValue());
        }
    }
}
```

Output

```
Top 3:
1. Aman - 3100
2. Dev - 3100
3. Isha - 2750
```

Part B — Project Work (Hour 2): MiniBank

Accounts move from a plain array to the Collections Framework: AccountStore keeps a HashMap keyed by account number, a history ArrayList, sorting with a Comparator, natural ordering through Comparable, and a ConcurrentHashMap inside TransactionProcessor for thread safety. Files not listed are unchanged from Practical 11.

4) Account implements Comparable<Account> — compareTo() by account number

Code

model/Account.java

```
package model;

import exception.BankException;
import exception.DailyLimitExceededException;
import exception.InsufficientFundsException;
import exception.InvalidAmountException;
import model.annotation.Id;
import model.annotation.MaxLength;
import model.annotation.Positive;

import java.io.Serializable;
import java.util.Objects;

public class Account implements Transactable, InterestBearing, Serializable, Comparable<Account> {
    private static final long serialVersionUID = 1L;
    public static final long DAILY_LIMIT = 10000;

    @Id
    @MaxLength(10)
    protected String accountNumber;
    @Positive
    protected long balance;
    private double rate;
    private transient long withdrawnToday;   // transient: NOT saved, resets to 0 after loading

    public Account(String accountNumber, long initialBalance, double rate) {
        this.accountNumber = accountNumber;
        this.balance = initialBalance;
        this.rate = rate;
    }

    // Thread-safe: only one thread at a time may run a synchronized method of this object
    @Override
    public synchronized void deposit(long amount) throws InvalidAmountException {
        if (amount <= 0) {
            throw new InvalidAmountException(amount);
```

```
        }
        balance += amount;
    }

    @Override
    public synchronized boolean withdraw(long amount)
            throws InsufficientFundsException, InvalidAmountException, DailyLimitExceededException {
        if (amount <= 0) {
            throw new InvalidAmountException(amount);
        }
        if (withdrawnToday + amount > DAILY_LIMIT) {
            throw new DailyLimitExceededException(DAILY_LIMIT);
        }
        if (amount > balance) {
            throw new InsufficientFundsException(amount - balance);
        }
        balance -= amount;
        withdrawnToday += amount;
        return true;
    }

    public void transfer(Account to, long amount) throws BankException {
        boolean done = false;
        try {
            withdraw(amount);
            to.deposit(amount);
            done = true;
        } catch (BankException e) {
            System.out.println("Transfer failed: " + e.getMessage());
            throw e;
        } finally {
            System.out.println("Transfer attempt " + accountNumber + " -> "
                    + to.getAccountNumber() + " finished, success=" + done);
        }
    }
    @Override
    public double interestRate() {
        return rate;
    }

    @Override
    public long getBalance() {
        return balance;
    }

    public long getWithdrawnToday() {
        return withdrawnToday;
    }

    public String getAccountNumber() {
        return accountNumber;
    }

    @Override
    public int compareTo(Account other) {
        return accountNumber.compareTo(other.accountNumber);   // natural order = by account number
    }

    @Override
    public String toString() {
        return "Account[" + accountNumber + ", balance=" + balance + "]";
    }

    @Override
    public boolean equals(Object o) {
        if (this == o) {
            return true;
        }
        if (!(o instanceof Account)) {
            return false;
        }
        Account other = (Account) o;
        return accountNumber.equals(other.accountNumber);
    }

    @Override
    public int hashCode() {
        return Objects.hash(accountNumber);
    }
}
```

1-3) AccountStore — HashMap, ArrayList history, sortedByBalance()

Code

service/AccountStore.java

```
package service;

import model.Account;

import java.util.ArrayList;
import java.util.Collection;
import java.util.Comparator;
import java.util.HashMap;
import java.util.TreeMap;

public class AccountStore {
    private final HashMap<String, Account> accounts = new HashMap<>();
    private final ArrayList<String> history = new ArrayList<>();

    public void add(Account account) {
        accounts.put(account.getAccountNumber(), account);
        history.add("ADDED " + account.getAccountNumber());
    }

    public Account get(String accountNumber) {
        history.add("LOOKUP " + accountNumber);
        return accounts.get(accountNumber);
    }

    public Collection<Account> getAll() {
        return accounts.values();
    }

    public ArrayList<String> getHistory() {
        return history;
    }

    /** Highest balance first. */
    public ArrayList<Account> sortedByBalance() {
        ArrayList<Account> list = new ArrayList<>(accounts.values());
        list.sort(Comparator.comparingLong(Account::getBalance).reversed());
        return list;
    }

    // supplementary: all accounts with balance above a limit
    public ArrayList<Account> balanceAbove(long limit) {
        ArrayList<Account> result = new ArrayList<>();
        for (Account a : accounts.values()) {
            if (a.getBalance() > limit) result.add(a);
        }
        return result;
    }

    // supplementary: TreeMap keeps keys in sorted order
    public TreeMap<String, Account> asSortedMap() {
        return new TreeMap<>(accounts);
    }
}
```

Supplementary problems covered: balanceAbove(limit) and asSortedMap() using a TreeMap.

5) TransactionProcessor — ConcurrentHashMap for the shared accounts

Code

service/TransactionProcessor.java

```
package service;

import model.Account;

import java.util.Map;
import java.util.concurrent.ConcurrentHashMap;
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;
import java.util.concurrent.TimeUnit;

public class TransactionProcessor {
    private final ExecutorService pool = Executors.newFixedThreadPool(4);
    // thread-safe map shared by all worker threads
    private final Map<String, Account> accounts = new ConcurrentHashMap<>();

    public void register(Account account) {
        accounts.put(account.getAccountNumber(), account);
    }

    public Account get(String accountNumber) {
        return accounts.get(accountNumber);
    }

    public int size() {
        return accounts.size();
    }

    public void submit(Runnable task) {
        pool.execute(task);
    }

    public void stop() throws InterruptedException {
        pool.shutdown();
        pool.awaitTermination(30, TimeUnit.SECONDS);
    }
}
```

6) main — look up, sort and process accounts

Code

service/MiniBank.java

```
package service;

import exception.BankException;
import model.Account;

public class MiniBank {
    public static void main(String[] args) throws Exception {
        AccountStore store = new AccountStore();
        store.add(new Account("AC0001", 5000, 0.04));
        store.add(new Account("AC0002", 2000, 0.04));
        store.add(new Account("AC0003", 9000, 0.04));

        System.out.println("--- get(\"AC0002\") ---");
        System.out.println(store.get("AC0002"));
        System.out.println("get(\"AC0009\") = " + store.get("AC0009"));

        System.out.println("--- sortedByBalance() (highest first) ---");
        for (Account a : store.sortedByBalance()) System.out.println(a);

        System.out.println("--- natural order: Collections.sort() uses compareTo() (by
            accountNumber) ---");
        var list = store.sortedByBalance();
        java.util.Collections.sort(list);
        for (Account a : list) System.out.println(a);

        System.out.println("--- balance above 4000 ---");
        for (Account a : store.balanceAbove(4000)) System.out.println(a);

        System.out.println("--- TreeMap keys ---");
        System.out.println(store.asSortedMap().keySet());

        System.out.println("--- Transaction history (ArrayList) ---");
        store.getHistory().forEach(h -> System.out.println("  " + h));

        System.out.println("--- Thread-safe ConcurrentHashMap in TransactionProcessor ---");
        TransactionProcessor tp = new TransactionProcessor();
```

```
        for (Account a : store.getAll()) tp.register(a);
        for (int i = 0; i < 100; i++) {
            tp.submit(() -> {
                try {
                    tp.get("AC0001").deposit(10);
                } catch (BankException e) {
                    System.out.println(e.getMessage());
                }
            });
        }
        tp.stop();
        System.out.println("Accounts in processor map: " + tp.size() + ", AC0001 after 100 deposits
            of 10: "
                + tp.get("AC0001").getBalance() + " (expected 6000)");
    }
}
```

Output

```
--- get("AC0002") ---
Account[AC0002, balance=2000]
get("AC0009") = null
--- sortedByBalance() (highest first) ---
Account[AC0003, balance=9000]
Account[AC0001, balance=5000]
Account[AC0002, balance=2000]
--- natural order: Collections.sort() uses compareTo() (by accountNumber) ---
Account[AC0001, balance=5000]
Account[AC0002, balance=2000]
Account[AC0003, balance=9000]
--- balance above 4000 ---
Account[AC0001, balance=5000]
Account[AC0003, balance=9000]
--- TreeMap keys ---
[AC0001, AC0002, AC0003]
--- Transaction history (ArrayList) ---
  ADDED AC0001
  ADDED AC0002
  ADDED AC0003
  LOOKUP AC0002
  LOOKUP AC0009
--- Thread-safe ConcurrentHashMap in TransactionProcessor ---
Accounts in processor map: 3, AC0001 after 100 deposits of 10: 6000 (expected 6000)
```

Package layout, build and tag

Code

```
minibank/
 |-- exception/
 |   |-- AccountNotFoundException.java
 |   |-- BankException.java
 |   |-- DailyLimitExceededException.java
 |   |-- InsufficientFundsException.java
 |   +-- InvalidAmountException.java
 |-- model/
 |   |-- annotation/
 |   |   |-- Id.java
 |   |   |-- MaxLength.java
 |   |   +-- Positive.java
 |   |-- Account.java
 |   |-- Customer.java
 |   |-- InterestBearing.java
 |   |-- Premium.java
 |   |-- PremiumAccount.java
 |   +-- Transactable.java
 |-- service/
 |   |-- AccountStore.java
 |   |-- MiniBank.java
 |   |-- ReportGenerator.java
 |   |-- StatePersister.java
 |   |-- TransactionLog.java
 |   |-- TransactionProcessor.java
 |   +-- WithdrawRule.java
 +-- util/
     |-- AnnotationValidator.java
     +-- Validator.java

javac -d out $(find exception model util service -name "*.java")
java -cp out service.MiniBank
cd out
jar --create --file minibank.jar --main-class=service.MiniBank .
java -jar minibank.jar
cd ..
```

	`git add .`
	`git commit -m "Practical 12 - collections framework"`
	`git tag lab-12`

get("AC0002") returns the correct account and sortedByBalance() prints the accounts in the order 9000, 5000, 2000. 100 concurrent deposits through the ConcurrentHashMap give exactly 6000.

Key Questions / Analysis / Interpretation

1. What is the difference between a List, a Set and a Map?

A List is an ordered collection that allows duplicates and access by index (ArrayList). A Set holds unique elements only (HashSet, LinkedHashSet, TreeSet). A Map stores key-value pairs with unique keys and looks values up by key (HashMap, TreeMap); it is not a Collection itself.

2. When do you use a HashMap versus a TreeMap?

HashMap gives average O(1) get/put with no ordering and is the default choice for look-ups such as account number to Account. TreeMap keeps keys sorted (O(log n) operations) and is used when sorted iteration or range queries are needed; its keys must be comparable.

3. What is the difference between Comparable and Comparator?

Comparable is implemented by the class itself (compareTo) and defines one natural order, here by account number. A Comparator is a separate object (compare) so a class can be sorted in many different ways, for example by balance descending, without changing the class.